

# Stock/Crypto Analytics Backend Project Roadmap

Tech stack: FastAPI, PostgreSQL, Celery + Redis, Docker + Docker Compose

## Phase 0: Bare API Skeleton

Get a single FastAPI app running locally with basic endpoints before adding any other infrastructure.

**Checkpoint:** GET /prices and GET /prices/{symbol} work against a fake in-memory list.

**Checkpoint:** You can explain what `app = FastAPI()` and a path decorator actually do, without looking it up.

## Phase 1: Real Database, No Workers Yet

Swap the fake Python list for real PostgreSQL, keeping endpoints and logic the same. This isolates learning SQLAlchemy/Postgres from Docker or Celery.

- Install Postgres locally (not in Docker yet) and connect via psql or a GUI tool like TablePlus.
- Write your first SQLAlchemy model (e.g. PriceTick: id, symbol, price, timestamp) and a database.py session setup.
- Set up Alembic and run your first migration.
- Rewrite /prices endpoints to query the real DB instead of the fake list.

**Checkpoint:** You can see the actual table exists in Postgres.

**Checkpoint:** Data survives a server restart.

## Phase 2: Manual Ingestion, No Automation Yet

Write a plain Python script (not an endpoint, not a Celery task) that fetches a real price from an API like CoinGecko and inserts it into Postgres. This isolates fetch+insert logic from scheduling.

- Add a POST endpoint that triggers this same function on demand.

**Checkpoint:** Running `python fetch_price.py` manually adds a new row visible in the DB.

**Checkpoint:** The function works identically whether called by script or API request.

## Phase 3: Background Tasks - Simple First

Use FastAPI's built-in BackgroundTasks before touching Celery. This teaches 'don't block the request' with zero new infrastructure. Only then add Redis and Celery.

- Install Redis locally, wrap your fetch function in a `@celery_app.task`, and run a Celery worker in a separate terminal.
- Add Celery Beat for scheduling (e.g. fetch prices every 5 minutes automatically).

**Checkpoint:** A POST endpoint returns instantly while the price-fetch happens after the response is sent.

**Checkpoint:** You trigger a task from an endpoint and watch it execute in the worker's terminal logs.

**Checkpoint:** Rows appear in Postgres automatically, without manual triggering.

## Phase 4: Analytics Endpoints

With real data flowing in automatically, build the actual analytics part of the API. Pure SQL/Python logic, no new infrastructure concepts.

- Endpoints like moving average, percent change over 24h, min/max over a date range.

**Checkpoint:** You can explain the SQL query behind each endpoint, not just call an ORM method blindly.

## Phase 5: Containerize Everything

Only now write the Dockerfile and docker-compose.yml, since every piece (API, worker, Redis, Postgres) already works independently. Docker's job is orchestration, not debugging.

- api service: runs FastAPI via uvicorn. Checkpoint: docker compose up serves /docs on localhost.
- worker service: runs Celery worker. Checkpoint: logs show tasks being picked up.
- redis service: message broker. Checkpoint: worker connects without manual Redis install.
- db service: Postgres. Checkpoint: data persists across docker compose down/up via a volume.

## Phase 6: Simple Frontend

Keep this minimal since the focus is backend - a single HTML page with fetch() calls, or a lightweight tool like Streamlit, just enough to visually confirm the API works end-to-end.

**Checkpoint:** A webpage displays a live chart or table pulling from your /analytics endpoint.

*Note: Each phase only starts once the previous phase's checkpoint is real and working. Resist jumping ahead to Docker or Celery before Phase 1's database is solid.*